

COS30018 - Option C - Task 3: Data Visualization Report

Chiraath Madahapola - 104834009

September 7, 2025

Contents

1	Executive Summary	3
2	Task Overview	4
2.1	Objectives	4
2.2	Requirements Analysis	4
3	Implementation Details	5
3.1	File Structure	5
3.2	Core Functions	5
3.2.1	1. display_candlestick_chart()	5
3.2.2	2. display_boxplot_chart()	6
3.3	Technical Implementation	6
3.3.1	Libraries Used	6
3.3.2	Data Processing	6
3.3.3	Visualization Features	6
4	Code Analysis	7
4.1	Key Challenges Addressed	7
4.1.1	1. MultiIndex Column Handling	7
4.1.2	2. Data Type Compatibility	7
4.1.3	3. Library Integration	7
4.1.4	4. Data Window Management	7
4.2	Code Quality Features	8
4.2.1	Simplicity and Clarity	8
4.2.2	Error Handling	8
4.2.3	Flexibility	8
5	Testing and Validation	9
5.1	Demo Function	9
5.2	Test Results	9
6	Usage Examples	11
6.1	Basic Usage	11
6.2	Advanced Parameters	11

7	Results and Insights	12
7.1	Visualization Capabilities	12
7.2	Practical Applications	12
8	Conclusion	13

Chapter 1

Executive Summary

This report documents the implementation of Task 3, which focuses on data visualization for financial stock market data. The task involves creating two key visualization functions: a candlestick chart using the mplfinance library and a boxplot chart for analyzing price movements over rolling windows. These visualizations enhance the project's analytical capabilities by providing clear graphical representations of stock price patterns and trends.

Chapter 2

Task Overview

2.1 Objectives

Task 3 aims to implement data visualization capabilities for the stock prediction project, specifically:

1. **Candlestick Chart Function:** Display OHLC (Open, High, Low, Close) data with volume information
2. **Boxplot Chart Function:** Show price distribution analysis using rolling windows
3. **Parameter Flexibility:** Allow customization of display parameters (`n_days`, `window_size`)
4. **Professional Presentation:** Create publication-ready visualizations

2.2 Requirements Analysis

Based on the task specification, the implementation must:

- Use the `mplfinance` library for candlestick charts
- Implement detailed parameter documentation
- Include an option for displaying specific numbers of trading days ($n \geq 1$)
- Provide boxplot analysis for moving windows of consecutive trading days
- Add comments explaining code functionality

Chapter 3

Implementation Details

3.1 File Structure

The Task 3 implementation consists of:

```
task3/
├── task3_visualization.py          # Main visualization functions
└── figures/                       # Generated visualization outputs
    ├── candlestick_chart_YYYYMMDD_HHMMSS.png
    └── boxplot_chart_YYYYMMDD_HHMMSS.png
```

3.2 Core Functions

3.2.1 1. display_candlestick_chart()

Purpose: Creates professional candlestick charts for financial data analysis

Key Features: - Uses mplfinance library for professional financial charting - Supports filtering by number of days (n_days parameter) - Displays OHLC data with volume information - Automatic title adjustment based on data range

Parameters: - data: DataFrame with OHLC columns - title: Chart title (default: "Stock Analysis") - n_days: Number of recent days to display (None for all data)

3.2.2 2. display_boxplot_chart()

Purpose: Analyzes price distribution using rolling window boxplots

Key Features: - Creates dual-panel visualization (timeline + boxplot) - Rolling window analysis for price distribution - Statistical insights for latest window - Customizable window size for different analysis periods

Parameters: - data: DataFrame with Close column - window_size: Rolling window size (default: 30 days) - title: Chart title (default: "Price Movement Analysis")

3.3 Technical Implementation

3.3.1 Libraries Used

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import mplfinance as mpf
import yfinance as yf
```

3.3.2 Data Processing

- Automatic MultiIndex column flattening for yfinance compatibility
- Data validation and NaN removal
- Rolling window generation for boxplot analysis
- Proper handling of datetime indices
- Robust error handling for missing or invalid data

3.3.3 Visualization Features

- Professional styling using mplfinance
- Color-coded candlesticks (green for bullish, red for bearish)
- Volume display below main price chart
- Grid lines and proper axis labeling
- Statistical annotations on boxplots
- Automatic figure saving to figures/ directory with timestamps
- High-resolution PNG output (300 DPI)

Chapter 4

Code Analysis

4.1 Key Challenges Addressed

4.1.1 1. MultiIndex Column Handling

Challenge: yfinance returns data with MultiIndex columns that mplfinance cannot process

Solution: Added automatic column flattening to extract level 0 column names

4.1.2 2. Data Type Compatibility

Challenge: Ensuring data types are compatible with mplfinance requirements **Solution:** Implemented proper data cleaning and NaN removal before visualization

4.1.3 3. Library Integration

Challenge: Integrating mplfinance with existing matplotlib workflows **Solution:** Used mplfinance for candlestick charts while maintaining matplotlib for boxplots

4.1.4 4. Data Window Management

Challenge: Creating meaningful rolling windows for boxplot analysis **Solution:** Implemented step-based window generation to avoid overcrowding

4.2 Code Quality Features

4.2.1 Simplicity and Clarity

- Clean, readable function signatures
- Minimal but effective commenting
- Straightforward parameter handling
- University-appropriate code style

4.2.2 Error Handling

- Input validation for required parameters
- Graceful handling of missing data
- Clear error messages for debugging

4.2.3 Flexibility

- Configurable display parameters
- Support for different data ranges
- Adaptable to various stock symbols

Chapter 5

Testing and Validation

5.1 Demo Function

The implementation includes a `demo_task3()` function that: - Downloads sample META stock data - Tests both visualization functions - Demonstrates parameter usage - Validates functionality

5.2 Test Results

Execution Output:

```
Task 3: Data Visualization Demo
```

```
Downloaded 250 records
```

```
Data columns: [('Close', 'META'), ('High', 'META'), ('Low', 'META'), ('Open', 'META'), ('Volume', 'META')]
```

```
Data shape: (250, 5)
```

```
Creating candlestick chart...
```

```
Candlestick chart saved to figures/candlestick_chart_20250907_213234.png
```

```
Creating boxplot chart...
```

```
Boxplot chart saved to figures/boxplot_chart_20250907_213236.png
```

```
Demo complete!
```

Key Achievements: - Successfully downloaded 250 records of META stock data (2023-2024)
- Handled MultiIndex columns from yfinance automatically - Created candlestick chart with 60-day window showing OHLC data - Generated boxplot analysis with 30-day rolling windows
- Both visualizations displayed correctly without errors - Automatically saved high-resolution

figures to `task3/figures/` directory - Generated timestamped PNG files for documentation and reporting - Code executed cleanly with proper data processing

Chapter 6

Usage Examples

6.1 Basic Usage

```
import yfinance as yf
from task3_visualization import display_candlestick_chart, display_boxplot_chart

# Download data
data = yf.download("META", start="2023-01-01", end="2024-01-01")

# Create candlestick chart
display_candlestick_chart(data, "META Stock Analysis", n_days=60)

# Create boxplot analysis
display_boxplot_chart(data, window_size=30, "META Price Movement")
```

6.2 Advanced Parameters

```
# Full dataset candlestick
display_candlestick_chart(data, "Complete Analysis", n_days=None)

# Short-term boxplot analysis
display_boxplot_chart(data, window_size=20, "Short-term Analysis")
```

Chapter 7

Results and Insights

7.1 Visualization Capabilities

The implemented functions provide:

1. **Market Trend Analysis:** Candlestick charts reveal bullish/bearish patterns
2. **Volatility Assessment:** Boxplots show price distribution and volatility periods
3. **Time-based Analysis:** Rolling windows capture changing market conditions
4. **Professional Presentation:** Publication-ready charts for reports

7.2 Practical Applications

- Technical analysis for trading decisions
- Risk assessment through volatility analysis
- Market trend identification
- Educational demonstrations of financial data patterns

Chapter 8

Conclusion

Task 3 successfully implements the required data visualization functionality with clean, efficient code. The candlestick and boxplot functions provide comprehensive analytical capabilities while maintaining simplicity and university-appropriate coding standards. The implementation demonstrates practical application of financial data visualization techniques using industry-standard libraries.

The visualization functions enhance the overall project by providing clear graphical insights into stock price patterns, supporting both technical analysis and educational objectives. The code is well-structured, documented, and ready for integration with the broader stock prediction system.